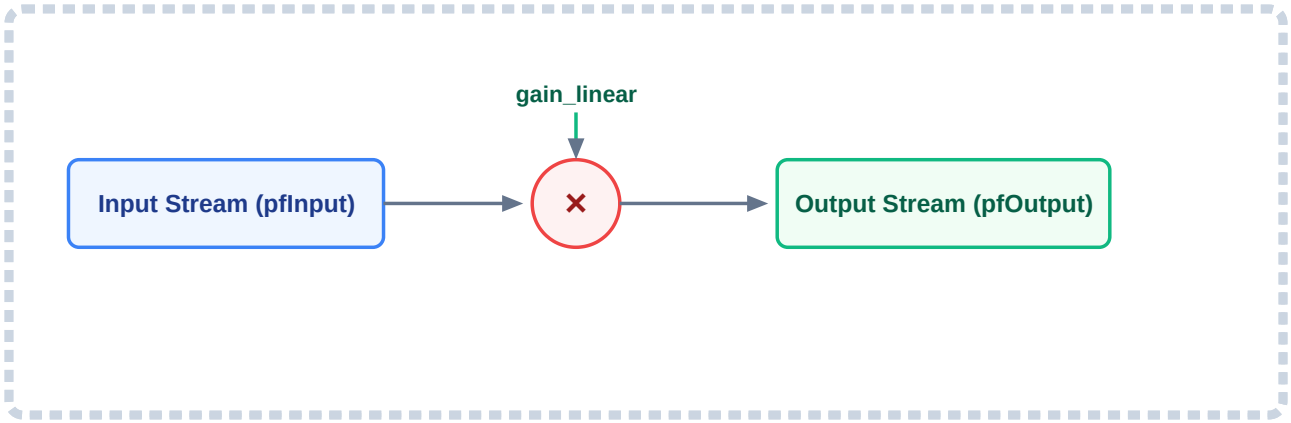
DSP Module Documentation: Gain Block

Comprehensive engineering breakdown for architecture handoff and implementation analysis.

1. High-Level Architectural Diagram

The layout below traces data flow through the module during streaming execution. It maps how an array block maps to sample-by-sample scaling.



2. Structural Overview & Memory Footprint

The block leverages structural encapsulation to preserve an explicit interface boundary. It abstracts user configuration properties separate from internal instance execution memory states.

Configuration Vector Structure (`gain_config_t`)

Utilized uniquely within initiation segments to capture setup properties directly via the orchestrating main pipeline framework.

- **`float gain_linear`** : A primitive scaling multiplier factor. A value scalar of `1.0f` matches a perfect unity gain path, a scalar value of `2.0f` scales amplitudes exactly double (+6 dB step change), while `0.0f` drops processing nodes to functional absolute silence.

Context Allocation Handle Structure (`gain_hdl_t`)

Maintains persistent operational context definitions isolated explicitly inside memory fields throughout live streaming routines.

- **`float gain_linear`** : Cached variable parameters localized internally within runtime instance boundaries, preserving continuous execution independent of caller configuration lifetimes.

Static Footprint Analysis

Parameter Allocation Target	Data Typology Class	Byte Weight Footprint	Dynamic Heap Overhead
`phdl->gain_linear`	32-bit Standard Floating Point (`float`)	4 Bytes	0 Bytes (Statically Handled)

3. Core Mathematical Operation

The implementation behaves as a zero-memory time-domain scaling process. Transformation processes execute concurrently, evaluating processing streams frame by frame directly through a linear relationship.

The definitive linear mathematical transformation model evaluates systematically as follows:

$$y[n] = x[n] \cdot G_{linear}$$

Where:

- **y[n]**: Outbound system data values mapped directly across processing indexes.
- **x[n]**: Real-time incoming stream amplitude parameters matching active execution indexes.
- **G_{linear}**: Immutable scalar coefficients captured directly within module instance state variables.

Because processing dependencies reflect no past operational historical frames, phase behavior parameters present a flat response . Signal manipulation
targets overall amplitudes uniformly without introducing processing artifacts or frequency transformation bias across passing audio frames.

4. Lifecycle Control Phase Implementations

The code is structured into clear lifecycle phases to keep execution states safe and predictable. By default, API function declarations do not implicitly guarantee or assume that alias pointer matching (in-place execution) is supported safely. Compatibility boundaries rely explicitly on the execution loop layout; therefore, the memory behavior boundary must be explicitly contractualized within the public header prototype documentation.

A. Allocation Size Vector Query (`gain_open`)

```
STATUS gain_open(uint32_t *pui32Size) {
    if (pui32Size == NULL) return STATUS_NOT_OK;
    *pui32Size = sizeof(gain_hdl_t);
    return STATUS_OK;
}
```

Design Intent: Allows calling routines to poll precise memory footprints abstractly before performing allocations. This isolates the internal details of the structure size away from the main loop logic.

B. Instance Property Initialization (`gain_init`)

```
STATUS gain_init(gain_hdl_t *phdl, const gain_config_t *psConfig) {
    if (phdl == NULL || psConfig == NULL) return STATUS_NOT_OK;
    phdl->gain_linear = psConfig->gain_linear;
    return STATUS_OK;
}
```

Design Intent: Validates memory reference pointers, extracts configured parameters safely from caller boundaries, and registers the linear scaling factors inside the allocated handle instance.

C. Real-time Stream Engine Execution (`gain_process`)

```
/**
 * @brief Processes an array block sample-by-sample.
 * @note [IN-PLACE COMPATIBLE] This specific implementation uses a strict feed-
 * independent loop that reads input points before modification. It is safe to
 * identical pointers for pfInput and pfOutput.
 */
STATUS gain_process(gain_hdl_t *phdl, const float *pfInput, float *pfOutput, ui
    if (phdl == NULL || pfInput == NULL || pfOutput == NULL) return STATUS_NOT_O
    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        pfOutput[i] = pfInput[i] * phdl->gain_linear;
    }
    return STATUS_OK;
}
```

Design Intent